Les Conteneurs sous Linux (1)

-Conteneur Web-

🎯 Objectifs :

- Découvrir l'outil **Podman** pour la gestion de conteneurs.
- Apprendre à exécuter un conteneur rootless avec un serveur web.
- Créer une page HTML simple servie par un conteneur.
- Automatiser le démarrage du conteneur avec **systemd**.

🧪 Exercice 1 : Serveur web rootless avec Podman

☑ 1. Installer le paquet podman

```
sudo dnf install -y podman
```

```
[ons@mylocalhost ~]$ sudo dnf install -y podman
[sudo] password for ons:
Updating Subscription Management repositories.
Last metadata expiration check: 0:35:25 ago on Mon 04 May 2026 09:24:12 AM CET.
Package podman-6:5.6.0-14.el9_7.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ons@mylocalhost ~]$
```

Explication :
`dnf` est le gestionnaire de paquets de Red Hat.
`-y` permet de répondre automatiquement "oui" aux questions.

☑ 2. Créer l'utilisateur `webadmin` avec mot de passe `web`

```
sudo useradd webadmin
echo "web" | sudo passwd --stdin webadmin
```

```
[ons@mylocalhost ~]$ sudo useradd webadmin
[ons@mylocalhost ~]$ echo "web" | sudo passwd --stdin webadmin
Changing password for user webadmin.
passwd: all authentication tokens updated successfully.
[ons@mylocalhost ~]$
```

Explication :

- `useradd` : création d'un utilisateur.
- `passwd --stdin` : assigne le mot de passe "web" sans interaction.

☑ 3. Se connecter en SSH avec `webadmin`

Si vous êtes sur la même machine :

```
su - webadmin
```

```
[ons@mylocalhost ~]$ su - webadmin
Password:
[webadmin@mylocalhost ~]$
```

Si depuis une autre machine :

```
ssh webadmin@<adresse_IP>
```

Remplacez `<adresse_IP>` par l'adresse de votre machine.

☑ 4. Chercher toutes les images `httpd`

```
podman search httpd
```

```
[webadmin@mylocalhost ~]$ podman search httpd
WARN[0000] The cgroupv2 manager is set to systemd but there is no systemd user session ava
ilable
WARN[0000] For using systemd, you may need to log in using a user session
WARN[0000] Alternatively, you can enable lingering with: `loginctl enable-linger 1005` (po
ssibly as root)
WARN[0000] Falling back to --cgroup-manager=cgroupfs
NAME                                                    DESCRIPTION
registry.redhat.io/rhscl/httpd-24-rhel7                 Platform for running A
pache httpd 2.4 or bui...
registry.redhat.io/rhel8/httpd-24                       Platform for running A
pache httpd 2.4 or bui...
registry.redhat.io/ubi8/httpd-24                        Platform for running A
pache httpd 2.4 or bui...
registry.redhat.io/rhel9/httpd-24                       Platform for running A
pache httpd 2.4 or bui...
registry.redhat.io/ubi9/httpd-24                        Platform for running A
pache httpd 2.4 or bui...
registry.redhat.io/rhel10/httpd-24                      Platform for running A
pache httpd 2.4 or bui...
registry.redhat.io/ubi10/httpd-24                       Platform for running A
pache httpd 2.4 or bui...
```

Objectif :

Cette commande sert à **rechercher des images de conteneurs** disponibles en ligne (sur des **registries**, c'est-à-dire des dépôts d'images) qui correspondent au mot-clé `httpd`.

Dans notre cas, on cherche des images qui contiennent un **serveur web Apache (httpd)**, afin de pouvoir les utiliser plus tard pour **créer un conteneur**.

📦 Où sont recherchées ces images ?

Par défaut, Podman cherche sur les **registries configurés** (souvent `docker.io`, parfois `registry.redhat.io` si c'est un système Red Hat Enterprise Linux).

📄 Exemple de sortie :

```
NAME                            DESCRIPTION                          STARS  OFFICIAL
docker.io/library/httpd         The Apache HTTP Server Project       4000   [OK]
docker.io/bitnami/httpd         Bitnami Apache container             150    No
quay.io/someuser/httpd-custom   Custom httpd image                   12     No
```

🧠 Pourquoi c'est utile ?

- Pour **vérifier** si une image existe.
- Pour **choisir** une image adaptée à vos besoins (officielle, personnalisée, légère, etc.).
- Pour **comparer** les images disponibles (description, source, etc.).

En résumé :

La commande `podman search httpd` permet de **trouver les images disponibles** en ligne pour ensuite en **télécharger une** avec la commande `podman pull`.

C'est un peu comme chercher une application dans un **app store** avant de la télécharger et de l'installer.

☑ 5. Télécharger l'image `docker.io/library/httpd`

```
podman pull docker.io/library/httpd
[webadmin@mylocalhost ~]$ podman pull docker.io/library/httpd
WARN[0000] The cgroupv2 manager is set to systemd but there is no systemd user session av
ilable
WARN[0000] For using systemd, you may need to log in using a user session
WARN[0000] Alternatively, you can enable lingering with: `loginctl enable-linger 1005` (p
ssibly as root)
WARN[0000] Falling back to --cgroup-manager=cgroupfs
WARN[0000] The cgroupv2 manager is set to systemd but there is no systemd user session av
ilable
WARN[0000] For using systemd, you may need to log in using a user session
WARN[0000] Alternatively, you can enable lingering with: `loginctl enable-linger 1005` (p
ssibly as root)
WARN[0000] Falling back to --cgroup-manager=cgroupfs
Trying to pull docker.io/library/httpd:latest...
Getting image source signatures
Copying blob 1935f25c7dc4 done   |
Copying blob 3531af2bc2a9 done   |
Copying blob 4f4fb700ef54 done   |
Copying blob d32ec5ee95cb done   |
Copying blob 5b16784cbecd done   |
Copying blob b267b545226b done   |
Copying config d20a6ad935 done   |
Writing manifest to image destination
d20a6ad9358ed372a1f4f7c4756b40499b2002023bb0924ef7c7c6fc963594c9
```

Explication :
`podman pull` télécharge l'image depuis Docker Hub (par défaut).

☑ 6. Afficher la liste des images en local

```
podman images
```

```
[webadmin@mylocalhost ~]$ podman images
WARN[0000] The cgroupv2 manager is set to systemd but there is no systemd user session ava
ilable
WARN[0000] For using systemd, you may need to log in using a user session
WARN[0000] Alternatively, you can enable lingering with: `loginctl enable-linger 1005` (po
ssibly as root)
WARN[0000] Falling back to --cgroup-manager=cgroupfs
WARN[0000] The cgroupv2 manager is set to systemd but there is no systemd user session ava
ilable
WARN[0000] For using systemd, you may need to log in using a user session
WARN[0000] Alternatively, you can enable lingering with: `loginctl enable-linger 1005` (po
ssibly as root)
WARN[0000] Falling back to --cgroup-manager=cgroupfs
REPOSITORY                TAG         IMAGE ID      CREATED      SIZE
docker.io/library/httpd   latest      d20a6ad9358e  12 days ago  120 MB
[webadmin@mylocalhost ~]$
```

Affiche les images téléchargées localement (nom, taille, date...).

📋 Exemple de sortie (résultat attendu) :

```
REPOSITORY                TAG         IMAGE ID      CREATED        SIZE
docker.io/library/httpd   latest      dabbfbe0c57b  2 weeks ago    138 MB
localhost/test-image      v1          a13b8d9a1e3e  3 days ago     200 MB
```

🧠 Explication des colonnes :

Colonne	Signification
REPOSITORY	Nom du dépôt ou source de l'image (ex : Docker Hub, local)
TAG	Étiquette associée à l'image (souvent `latest` par défaut)
IMAGE ID	Identifiant unique de l'image (utilisé pour d'autres commandes)
CREATED	Date de création ou de téléchargement de l'image
SIZE	Taille occupée sur le disque

☑ 7. Inspecter l'image

```
podman inspect httpd
```

```
[
     {
          "Id": "d20a6ad9358ed372a1f4f7c4756b40499b2002023bb0924ef7c7c6fc963594c9",
          "Digest": "sha256:eeeb5c6dd40afcd20fe975babd9e0f75311fb4ae6a2fd473ea717a04b96d66
c4",
          "RepoTags": [
               "docker.io/library/httpd:latest"
          ],
          "RepoDigests": [
               "docker.io/library/httpd@sha256:89a0b59e6d7285f00fc8df952a1579c31eb035cbdcc
b96690051e8bb6432cbea",
               "docker.io/library/httpd@sha256:eeeb5c6dd40afcd20fe975babd9e0f75311fb4ae6a2
fd473ea717a04b96d66c4"
          ],
          "Parent": "",
          "Comment": "debuerreotype 0.17",
          "Created": "2026-04-22T01:22:01.942217074Z",
          "Config": {
               "ExposedPorts": {
                    "80/tcp": {}
               },
```


Fournit des détails techniques sur l'image (couches, variables, volumes...).

Exemple de sortie (résultat attendu) :


```
[
    {
        "Id": "dabbfbe0c57b...",
        "RepoTags": [
            "docker.io/library/httpd:latest"
        ],
        "Created": "2024-04-10T12:00:00Z",
        "Size": 138000000,
        "Config": {
            "Env": [
                "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin",
                "HTTPD_PREFIX=/usr/local/apache2"
            ],
            "ExposedPorts": {
                "80/tcp": {}
            },
            "Entrypoint": [
                "httpd-foreground"
            ]
        },
        "RootFS": {
            "Layers": [
                "sha256:aa1b...",
                "sha256:bb2c...",
                "sha256:cc3d..."
            ]
        }
    }
]
```


---

8. Lancer un conteneur rootless nommé `web` sur le port 8081

Contexte général :

Tu veux **lancer un conteneur Apache (httpd)** avec **Podman** en mode **rootless**, c'est-à-dire **avec un utilisateur non-root** (`webadmin`).
Ce conteneur servira une page web sur le port **8081** de ta machine (l'hôte).
Et tu veux que les fichiers HTML soient stockés dans un dossier spécifique sur ta machine (`/home/webadmin/web`).

— Préparer le dossier local (sur l'hôte) :

```
mkdir -p /home/webadmin/web
```

```
[webadmin@mylocalhost ~]$ mkdir -p /home/webadmin/web
[webadmin@mylocalhost ~]$
```

- **But :** créer un dossier **web** dans le dossier personnel de l'utilisateur `webadmin`.

- **Pourquoi ?**

Ce dossier servira à **stocker les fichiers web (HTML, images, etc.)** sur **ta machine physique**.
Il sera "partagé" avec le conteneur, donc quand tu modifies un fichier ici, c'est visible dans le conteneur.

— Lancer le conteneur Apache :

```
podman run -dit --name web -p 8081:80 -v /home/webadmin/web:/usr/local/apache2/htdocs:Z httpd
```

```
[webadmin@mylocalhost ~]$ podman run -dit --name web -p 8081:80 -v /home/webadmin/web:/usr
/local/apache2/htdocs:Z httpd
WARN[0000] The cgroupv2 manager is set to systemd but there is no systemd user session ava
ilable
WARN[0000] For using systemd, you may need to log in using a user session
WARN[0000] Alternatively, you can enable lingering with: `loginctl enable-linger 1005` (po
ssibly as root)
WARN[0000] Falling back to --cgroup-manager=cgroupfs
WARN[0000] The cgroupv2 manager is set to systemd but there is no systemd user session ava
ilable
WARN[0000] For using systemd, you may need to log in using a user session
WARN[0000] Alternatively, you can enable lingering with: `loginctl enable-linger 1005` (po
ssibly as root)
WARN[0000] Falling back to --cgroup-manager=cgroupfs
a22ff5d9ee6fb6d32f034a59b4cfc587eb53ff31980a18a9c799227a32040e48
```

```
CONTAINER ID  IMAGE                           COMMAND           CREATED         STATUS
     PORTS                 NAMES
a22ff5d9ee6f  docker.io/library/httpd:latest  httpd-foreground  31 seconds ago  Up 32 seco
nds  0.0.0.0:8081->80/tcp  web
[webadmin@mylocalhost ~]$
```

On la décompose mot par mot 👇

Élémenhtt	Explication
`Podman`	Outil utilisé pour gérer les conteneurs. Alternative à Docker.

Élémenhtt	Explication
`Run`	Démarre un nouveau conteneur.
`-d`	Détaché (detached mode) → le conteneur tourne en arrière-plan.
`-i`	Interactif → permet d'interagir avec le conteneur (utile avec un terminal).
`-t`	Terminal → donne un pseudo-terminal, utile pour des commandes bash par ex.
`--name web`	Donne un **nom** au conteneur : ici, on l'appelle `web`.
`-p 8081:80`	Mappe le **port 80 du conteneur** (port par défaut d'Apache) vers le **port 8081 de l'hôte**. Donc tu accèdes au site avec `http://localhost:8081`
`-v /home/webadmin/web:/usr/local/apache2/htdocs`	Monte le dossier `/home/webadmin/web` de l'hôte **dans** le conteneur à l'endroit où Apache cherche les fichiers HTML.
`Httpd`	C'est l'image que tu veux utiliser (Apache HTTP Server).

🖼️ Visualisation rapide :

```
+-----------------------------+                +--------------------------+
|   Hôte (webadmin)           |                |  Conteneur "web"         |
|                             | <----->        |                          |
| /home/webadmin/web          |   ↔ Volume     | /usr/local/apache2/htdocs |
+-----------------------------+                +--------------------------+

Port 8081 (host)  <---->  Port 80 (conteneur)
```

Donc si tu ouvres un navigateur et tapes :

📌 `http://localhost:8081`

Tu verras la page `index.html` que tu auras mise dans `/home/webadmin/web`.



🧠 Objectif global :

Tu mets en place un petit **serveur web Apache dans un conteneur**, mais :

- Tu **n'as pas besoin de droits root** grâce au **rootless Podman**.
- Tu gardes tous les fichiers **accessibles depuis l'hôte** (ce qui est pratique).
- Tu exposes ton serveur localement via le port 8081.

☑ 9. Afficher la liste des conteneurs

```
podman ps -a
CONTAINER ID  IMAGE                           COMMAND           CREATED        STATUS
   PORTS                 NAMES
a22ff5d9ee6f  docker.io/library/httpd:latest  httpd-foreground  8 minutes ago  Up 8 minute
s  0.0.0.0:8081->80/tcp  web
[webadmin@mylocalhost ~]$
```

Affiche tous les conteneurs (actifs ou non), avec leur statut.

podman start web (en cas ou le conteneur n'est pas actif)

☑ 10. Créer un fichier `index.html` dans le conteneur

Méthode 1 : en ligne de commande

```
echo "Bonjour Tekup" > /home/webadmin/web/index.html
[webadmin@mylocalhost ~]$ echo "Bonjour Tekup" > /home/webadmin/web/index.html
[webadmin@mylocalhost ~]$
```

Le fichier sera directement accessible depuis le conteneur grâce au volume partagé.

Vérification (optionnel) :

```
curl http://localhost:8081
```



☑ 11. Générer un service systemd pour le conteneur

Qu'est-ce que `systemd` ?

`systemd` est le **gestionnaire de services** utilisé par la majorité des distributions Linux modernes (Red Hat, Fedora, Ubuntu...).

🔧 Il s'occupe de :

- **Démarrer les services** au démarrage de l'ordinateur (ex : réseau, sshd, Apache…)
- **Arrêter/reprendre** ces services
- **Vérifier leur état**
- **Automatiser** certaines tâches liées à des processus

Un **service systemd**, c'est tout simplement **un fichier `.service`** qui décrit **comment lancer un programme automatiquement**, comme un conteneur Podman dans notre cas.

Objectif ici :

Tu veux que ton **conteneur Podman (`web`) soit lancé automatiquement au démarrage de la machine**, sans devoir retaper la commande `podman run ...`.

Donc, tu demandes à Podman de **générer un fichier de service systemd prêt à l'emploi**.

Commande à exécuter :

```
podman generate systemd --name web --files --restart-policy=always
```

```
[webadmin@mylocalhost ~]$ podman generate systemd --name web --files --restart-policy=alwa
ys
WARN[0000] The cgroupv2 manager is set to systemd but there is no systemd user session ava
ilable
WARN[0000] For using systemd, you may need to log in using a user session
WARN[0000] Alternatively, you can enable lingering with: `loginctl enable-linger 1005` (po
ssibly as root)
WARN[0000] Falling back to --cgroup-manager=cgroupfs
WARN[0000] The cgroupv2 manager is set to systemd but there is no systemd user session ava
ilable
WARN[0000] For using systemd, you may need to log in using a user session
WARN[0000] Alternatively, you can enable lingering with: `loginctl enable-linger 1005` (po
ssibly as root)
WARN[0000] Falling back to --cgroup-manager=cgroupfs

DEPRECATED command:
It is recommended to use Quadlets for running containers and pods under systemd.

Please refer to podman-systemd.unit(5) for details.
/home/webadmin/container-web.service
```

🕮 Explication détaillée :

Option	Signification
`generate systemd`	Demande à Podman de générer un fichier de type service systemd
`--name web`	Précise le nom du conteneur à prendre en compte (ici, `web`)
`--files`	Crée un **fichier `.service`** dans le **dossier courant**
`--restart-policy=always`	Ajoute une directive pour **relancer automatiquement** le service si le conteneur s'arrête (crash, redémarrage, etc.)

🎁 Résultat :

Un fichier comme celui-ci est généré :

```
container-web.service
```

Ce fichier est un= **descripteur systemd** qui permet de gérer ton conteneur comme n'importe quel autre service Linux :

- Tu peux le **démarrer**, **arrêter**, **activer** au démarrage.
- Il fonctionne **sans que tu aies à lancer manuellement Podman** à chaque fois.

☑ 12. Démarrer et activer le service systemd

Déplacer le fichier dans le bon répertoire :

```
mkdir -p ~/.config/systemd/user
mv container-web.service ~/.config/systemd/user/
[webadmin@mylocalhost ~]$ mkdir -p ~/.config/systemd/user
[webadmin@mylocalhost ~]$ mv container-web.service ~/.config/systemd/user/
[webadmin@mylocalhost ~]$
```

```
#loginctl enable-linger webadmin
```

Activer le service au démarrage :

```
systemctl --user daemon-reexec
systemctl --user daemon-reload
systemctl --user enable --now container-web.service
```

```
[webadmin@mylocalhost ~]$ podman ps
CONTAINER ID  IMAGE                           COMMAND           CREATED        STATUS
   PORTS                 NAMES
e5fac03fc386  docker.io/library/httpd:latest  httpd-foreground  3 minutes ago  Up 7 second
s  0.0.0.0:8081->80/tcp  web
[webadmin@mylocalhost ~]$
```

Active et démarre immédiatement le service.

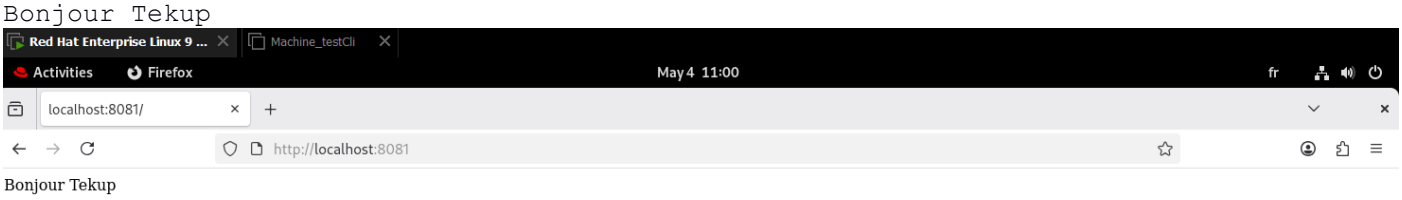
☑ 13. Tester la page web

Dans un navigateur web ou avec `curl` :

```
curl http://localhost:8081
```

Résultat attendu :

```
Bonjour Tekup
```



☑ 14. Redémarrer la machine et tester de nouveau

Redémarrage :

```
sudo reboot
```

Après redémarrage, reconnectez-vous et testez sur Firefox :

http://localhost:8081



sur teminal :

```
curl http://localhost:8081
```

🎯 **But :** vérifier que `systemd` relance automatiquement le conteneur.

```
[webadmin@mylocalhost ~]$ curl http://localhost:8081
Bonjour Tekup
[webadmin@mylocalhost ~]$
```

☑ 15. Supprimer le conteneur

```
systemctl --user disable --now container-web.service
```

```
[webadmin@mylocalhost ~]$ systemctl --user disable --now container-web.service
Removed "/home/webadmin/.config/systemd/user/default.target.wants/container-web.service".
[webadmin@mylocalhost ~]$
```

```
podman stop web
podman rm web
```

```
[webadmin@mylocalhost ~]$ podman stop web
web
[webadmin@mylocalhost ~]$ podman rm web
web
[webadmin@mylocalhost ~]$
```

☑ 16. Supprimer l'image

```
podman rmi httpd
```

```
[webadmin@mylocalhost ~]$ podman rmi httpd
Untagged: docker.io/library/httpd:latest
Deleted: d20a6ad9358ed372a1f4f7c4756b40499b2002023bb0924ef7c7c6fc963594c9
[webadmin@mylocalhost ~]$
```

Résumé des commandes clés

Action	Commande principale
Installer Podman	`sudo dnf install podman`
Créer utilisateur	`sudo useradd webadmin`
Chercher une image	`podman search httpd`
Télécharger une image	`podman pull httpd`
Lancer un conteneur	`podman run -dit --name web ...`

Action	Commande principale
Voir les conteneurs	`podman ps -a`
Créer un fichier HTML	`echo "Bonjour Tekup" > ...`
Générer service systemd	`podman generate systemd --name web --files`
Activer service	`systemctl --user enable --now container-web.service`
Supprimer conteneur/image	`podman stop/rm/rmi`